Plan: check all code logically

AI Invoice Assistant · 15/10/2026 · reading and reasoning, plus small Python checks without a bot

Goal: find logic errors (wrong result, missing check, rule broken, dead code), not style.
Not in scope: what the bots understand (that is in `tests/test_plan.md`).
Rules:

- Only a **copy** of the database (seed_data test DB, like `guard_tests.py`). Never `data/invoices.db`.
- Findings go into `tests/open_problems.md`, section A.
- No fixes during the check. Each fix comes later: one function, shown first, then your OK (CLAUDE.md).

1. Order: follow the data, bottom to top

#	File	Lines	Why in this order
1	`core/storage.py`	77	Everything else reads and writes through it
2	`entities/customers.py`	112	Simplest entity, the pattern for the other two
3	`entities/products.py`	131	Prices, used on invoices
4	`entities/invoices.py`	212	Needs customers and products, dates, status
5	`core/reports.py`	83	Turnover: paid only, by due date
6	`chat/bot.py`	120	Parsing commands and the guards
7	`chat/router.py`	68	Routing, sessions, which form opens
8	`chat/assistant.py`	408	Biggest file: tools, form_turnover, proposals
9	`api.py`	309	REST endpoints must use the same checks as the chat
10	`static/app.js`	485	Form messages must match what Python parses
11	`chat/guide.py`	74	Help (RAG), outside the data flow

2. Method for each function (same 4 steps)

Step	What
1. Read	What does it promise? (docstring, requirement)
2. Edge inputs	2–4 tricky inputs: empty, None, wrong type, border date, an id that does not exist, a duplicate
3. Try it	Call the function directly in Python on the test DB. No bot, so the result is the same every time.
4. Note	OK, or a finding: *input → got → expected*

3. Checklist per file

3.1 core/storage.py

Function	Check
`create_tables`	• NOT NULL on the right columns (buy_price may be NULL) • SQLite does **not enforce** REFERENCES by default, so is every link checked in Python?
`load`	• invoice lines sorted by `line` • an invoice without lines gives `[]`

`save`	• deletes the whole table, then writes it again: is it one transaction (an error halfway loses nothing)? • saving products while invoices point to them
`next_id`	• after deleting the **highest** id, is that number given again? (e.g. INV-012 deleted, then a new INV-012) • above 999: `CUS-1000`
`find`	• case: `cus-001` vs `CUS-001`

3.2–3.4 entities (customers, products, invoices)

Function	Check
`check_customer`, `check_product`, `check_invoice`	• empty strings and spaces only (" ") • price: negative, 0, "abc", "4,50" (comma) • invoice: due date before date, quantity 0 / 1.5 / "2", unknown customer or product, no lines
`to_price`, `to_date`, `to_quantity`	• 31/02/2026, 1/1/2026, 2026-01-01, a leap day • rounding: 0.1 + 0.2
`create_*`, `update_*`	• does update check the **merged** record, not only the new fields? • update with an unknown id • product price change: old invoices keep their `unit_price`
`delete_*`	• customer with invoices, product on an invoice, paid invoice: all refused • known #3: paid → unpaid → delete. Is that allowed? (your decision)
`find_customers`, `find_products`, `find_one`, `resolve`	• exact match before a partial match • two customers with the same name • "Lamp" when "Desk Lamp" and "Lamp Orbis" exist
`check_command`	• every command (CREATE/UPDATE/DELETE/SHOW) gives an error or None, never a crash • missing keys in `data`
`build_proposal`, `answer`	• the proposal shows exactly what will be saved (same values, same total) • `invoice_total` = sum of quantity × unit_price • `LIST_LIMIT`: what happens with 11+ matches?

3.5 core/reports.py

Function	Check
`paid_invoices`	• only `paid`, due date inside the period, both ends included • start = end (one day)
`turnover_by_customer`	• with a product: only that product's lines • a customer deleted later: `find(...)["name"]` crashes on None?
`turnover_by_product`	• with a customer: only that customer's invoices • a product deleted later: same crash question

`turnover_by_month`	• always 12 rows; the sum of the months = the turnover for the whole year • customer + product together
`check_period`, `period`	• month 13, month "3", year only, no year (= this year) • February in a leap year
Cross-check	Sum by customer = sum by product = sum of the 12 months (same period, no filter)

3.6 chat/bot.py

Function	Check
`parse_command`, `unique_keys`	• two commands in one reply • a value with a quote or a comma • a key given twice
`invented_target`	• known G6: substring check, so "Lamp" passes for "Desk Lamp" • an id written in lower case by the user
`invented_period`	• "2026" counts as written when the user wrote 26/10/2026. Correct? • "1/3/2031" vs "01/03/2031"
`clean_command`	• "null", "None", "N/A", "-", "?"
`converse`	• every path returns either a proposal or a reply, **never a save** • `open` is right on each path • `invented_period` is called twice (same result?)

3.7 chat/router.py

Part	Check
`route`	• no "route:" in the reply → it takes the first matching word of the whole reasoning • "question" and an entity in the same answer
`NEGATIVE_PRICE`	• known G3: "USB-3 Cable" • a date range "1-5" • runs before the bot even when the product bot is waiting
Sessions	• a waiting bot gets **every** next message, even a new topic ("turnover by customer") • an unknown `conversation_id` starts a new session • sessions are never removed (memory grows)
`ui` phrases	• every phrase must match a real question from the Modelfiles • invoice: the word "customer" in an error message also opens the customer chooser

3.8 chat/assistant.py

Part	Check
`TOOLS`, `UI_TOOLS`, `FORM_FOR`	• known #4: outdated descriptions; `get_turnover_by_product` has no `customer_id` • every tool name in TOOLS has a branch in `read` or `propose`
`period_from_user`	known #10: unused, so is it dead code?
`filter_id`, `NO_FILTER`	• an id that does not exist, so does it fall back to all or to an error?
`read`, `list_invoices`	• each tool gives the same numbers as reports.py • status synonyms (open, pending, settled)
`form_turnover`	• every message `app.js` can send is parsed (G11–G17 + all combinations) • a name with a comma or brackets ("Müller (CUS-004)")
`propose`	• uses `CHECKS`, so does a wrong tool argument give an error, not a proposal? • never saves

`converse`	• known #2: `MAX_ROUNDS` limits rounds, not calls • "show me " prefix: is it added only once? • what is returned when the bot gives no text

3.9 api.py

Part	Check
REST CRUD	• POST/PUT/DELETE run the **same** checks as the chat (e.g. a paid invoice gives HTTP 400) • 404 for an unknown id, 400 for bad data • PUT with an id in the body that differs from the path
`/reports/*`	• bad dates give 400, not 500 • same result as `form_turnover`
Chat endpoints	• `/chat`, `/customers/chat`, … : are the old per-entity endpoints still needed?
Yes path	• the only place where a proposal is saved: does it check again before saving? (the data may have changed in between)

3.10 static/app.js · 3.11 chat/guide.py

Part	Check
Form messages	• table: each form → the exact text it sends → the Python function that reads it • empty fields mean "all"
`showProposal`	• Yes calls the right endpoint and method; No saves nothing • the tabs refresh after a save
Dates	• `toDisplayDate`: ISO to DD/MM/YYYY, no time zone shift
`guide.py`	• the index is built once, so a guide change needs a restart • Ollama is not running: error message instead of a crash

4. Rules that cross all files (invariants)

#	Rule	How to check
I1	Nothing is saved without the Yes button	search every call of `storage.save`; each one must be reached only from a REST write endpoint
I2	Turnover = paid only, by due date	every turnover path goes through `paid_invoices`
I3	Chat and REST use the same checks	table: rule → function → called from chat? from REST?
I4	No dangling references	no invoice without a customer, no line without a product
I5	Prices ≥ 0, quantities whole and > 0, due date ≥ date	every entry point (chat, REST, assistant proposal)
I6	Ids are unique and never reused	`next_id` after deletes
I7	No dead code	every function is called somewhere (grep)

5. Output

Where	What
`tests/open_problems.md` A	one row per finding: *file:line · input → got → expected · possible fix*

`tests/guard_tests.py`	each finding that Python can show becomes a new G-case (shown to you first)
Short report	per file: OK / number of findings; the known ones (#2, #3, #4, #10, G3, G6) are confirmed, not counted again

6. Steps

Session	Files
1	storage, customers, products, invoices + I4, I5, I6
2	reports, bot, router + I1, I2
3	assistant, api + I3
4	app.js, guide + I7, then the findings list